



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Facultad de Ingeniería

Ingeniería de Software I

Profesor: Oscar Eduardo Álvarez Rodríguez

Integrantes:

Santiago Herrera Parra

Nael Nehuen Fontaine

David Santiago Vargas Parra

Daniel Alejandro Gomez Mancilla

Laura Sophia Castro Amaya

2026-1

Introducción

En el desarrollo moderno de software, las aplicaciones backend cumplen un papel fundamental al encargarse de la lógica del sistema, la gestión de datos y la comunicación con bases de datos.

El presente tutorial tiene como propósito guiar, paso a paso, la construcción de una aplicación backend utilizando el framework Django. A lo largo del proceso se desarrollará una aplicación básica tipo “Hola Mundo”, pero estructurada bajo buenas prácticas, incluyendo la configuración del entorno, la conexión a una base de datos, el uso del ORM, la definición de entidades y la exposición de un endpoint funcional.

Objetivos

Objetivo general: Construir una aplicación backend funcional utilizando Django que permita la interacción con una base de datos mediante un endpoint básico.

Objetivos específicos:

- Configurar el entorno de desarrollo para una aplicación Django
- Comprender la estructura de un proyecto backend
- Configurar y conectar una base de datos
- Utilizar el ORM de Django
- Crear una entidad y persistir datos
- Crear un endpoint funcional.
- Validar la aplicación mediante Postman.

Conceptos clave

Backend: Lógica del servidor que procesa datos y responde solicitudes.

Framework: Estructura que facilita el desarrollo de software.

ORM: Herramienta que permite interactuar con la base de datos mediante objetos.

Modelo: Representación de una tabla en la base de datos.

Migraciones: Mecanismo para crear y modificar estructuras en la base de datos.

Endpoint: Ruta que permite acceder a funcionalidades del backend.

Pre-requisitos

- Python 3.10 o superior.
- Git.
- Docker y Docker Compose.
- Postman.

Lenguaje y framework

Lenguaje: Python.

Framework: Django.

Librería ORM utilizado

Se utiliza Django ORM, el cual permite manipular la base de datos sin escribir SQL directamente.

Creación del entorno de desarrollo

Creación entorno virtual

```
python -m venv venv
```

- Activar Windows

```
venv\Scripts\activate
```

Instalar Django

```
pip install django
```

Creación del proyecto

```
django-admin startproject backend  
cd backend
```

Creación de la aplicación

```
python manage.py startapp api
```

Registrar la aplicación en backend/settings.py

```
INSTALLED_APPS = [  
    'api',  
]
```

Configuración de la base de datos

Django utiliza SQLite por defecto

En settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

Migraciones iniciales

Crea la base de datos automáticamente

```
python manage.py makemigrations  
python manage.py migrate
```

Definición de la entidad

En api/models.py:

```
from django.db import models  
  
class Persona(models.Model):  
    nombre = models.CharField(max_length=100)  
    edad = models.IntegerField()  
    def __str__(self):  
        return self.nombre
```

Migraciones del modelo

```
python manage.py makemigrations  
python manage.py migrate
```

Creación de la vista

En api/views.py:

```
from django.http import JsonResponse  
from .models import Persona  
  
def hola_mundo(request):
```

```
persona = Persona.objects.create(nombre="Ejemplo", edad=20)
return JsonResponse({
    "mensaje": "Hola Mundo",
    "nombre": persona.nombre,
    "edad": persona.edad
})
```

Configuración de rutas

Crear api/urls.py:

```
from django.urls import path
from .views import hola_mundo

urlpatterns = [
    path('hola/', hola_mundo),
]
```

Modificar backend/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

Ejecución del servidor

```
python manage.py runserver
```

- Acceder en el navegador

<http://127.0.0.1:8000/api/hola/>

Prueba con Postman

Abrir Postman y configurar con el método GET y la URL anterior

Respuesta:

```
{
```

```
"mensaje": "Hola Mundo",  
"nombre": "Ejemplo",  
"edad": 20  
}
```

Archivo .yaml (Docker Compose)

Archivo Docker-compose.yaml:

```
version: '3.9'  
  
services:  
  web:  
    build: .  
    command: python manage.py runserver 0.0.0.0:8000  
    volumes:  
      - ./app  
    ports:  
      - "8000:8000"
```

Configuración de contenedor (Dockerfile)

```
FROM python:3.10  
  
WORKDIR /app  
  
COPY . .  
  
RUN pip install django  
  
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```